

课程笔记

CS224R 第 13 讲: Meta Reinforcement Learning

Codex 基于 Stanford CS224R 2025 春季课程视频、字幕与官方 slides 重写

2026-04-13



视频作者/频道: Stanford Online

发布日期: 2025 年 12 月 08 日

视频时长: 1:09:10

视频链接: <https://www.youtube.com/watch?v=wSiyEpvoGkA>

目录

1	从多任务学习到 Meta-RL	2
1.1	本章小结	2
2	问题形式化与任务分布	2
2.1	本章小结	4
3	Black-box Meta-RL 的做法	4
3.1	本章小结	5
4	Meta-RL 中的探索难题	5
4.1	本章小结	7
5	总结与延伸	7

1 从多任务学习到 Meta-RL

这一讲先回顾了上一周的两条主线：多任务强化学习与目标条件强化学习。多任务 RL 把状态扩展为 $(\bar{s}, z_i)$ ，其中 z_i 是任务描述符；目标条件 RL 则把任务描述符具体化成目标状态 s_g 。这两类方法都在回答一个问题：**能否让一个策略不只学会一件事，而是学会一类相关任务？**

讲者随后把视角推进一步：如果面对一个从未见过的新任务，人类往往只需要少量尝试就能快速适应，而标准 RL 往往需要百万级 trial。以“使用一台新意式咖啡机”为例，机器人从零开始用 PPO 学习会非常慢，但人类会利用此前关于咖啡制作、工具操作和语义指令的经验。Meta-RL 正是试图把这种“利用既有经验进行快速适应”的能力形式化。

Meta-RL 的核心区别

它不是只要求一个策略在许多训练任务上表现好，而是要求它在**看到少量新任务经验之后**，能快速决定下一步该怎么做。

本讲也系统比较了三种迁移范式：

- 前向迁移：先在源任务训练，再在目标任务微调。这要求源任务与目标任务非常相似。
- 多任务迁移：在许多任务上同时训练，希望借助结构化任务描述实现零样本或低样本迁移。
- 元学习：训练时就显式考虑“适应”这件事，把少量上下文经验本身当作任务描述。

在 RL 场景里，Meta-RL 与多任务学习最大的差别是：多任务方法通常把任务 ID、目标状态或 prompt 直接喂给策略；Meta-RL 则把一小段轨迹、奖励和交互经验当作“上下文”，让策略根据这些经验推断任务。

为了建立直觉，讲者还对照了非 RL 领域的 few-shot learning。无论是少样本图像分类还是 LLM 的 in-context learning，本质上都在训练一个函数：给定少量训练样本，再对新输入做判断。Meta-RL 正是在 MDP 环境里做同样的事，只是“训练样本”变成了几段交互轨迹。

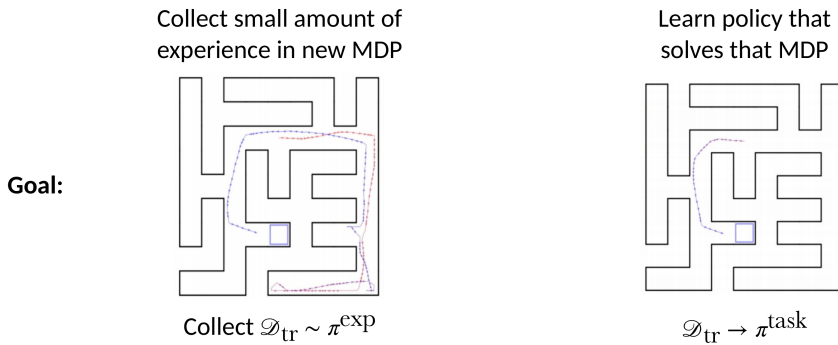
1.1 本章小结

本章的作用是把 Meta-RL 放回更大的迁移学习版图中。它既不同于单纯 fine-tuning，也不同于普通多任务策略，而是把“少量试错后快速适应”本身作为训练目标。

2 问题形式化与任务分布

在直观例子之后，课程给出一个更正式的表述。Meta-RL 不是只关心单个 MDP，而是假设存在一个任务分布 $p(\mathcal{M})$ ，每次训练或测试都要从这个分布中抽取一个具体 MDP。训练阶段的目标，是在许多训练任务上学会如何利用少量经验来适应；测试阶段的目标，是在一个新但同分布的任务上做出高效适应。

Meta-RL Example: Maze Navigation



Exploration-exploitation trade-off is a unique part of meta-RL!

diagram adapted from Duan et al. '17

图 1: Meta-RL 的基本流程：先在新任务中收集少量经验，再利用这段经验求解当前 MDP。

课程用迷宫导航作为核心例子。对于一个新迷宫，代理并不知道目标在哪里，所以先要做一段探索，得到训练上下文 $\mathcal{D}_{\text{tr}}$ ；之后再根据这段上下文在同一个迷宫中更快地找到目标。这个例子清楚地展示出 Meta-RL 与普通 RL 的差别：**探索本身不是最终目标，而是为了更快适应当前任务。**

更正式地，可以把 black-box Meta-RL 的策略写成

$$\pi_{\text{task}}(a_t \mid s_t, \mathcal{D}_{\text{tr}}).$$

这里：

- π_{task} 表示经过元训练后得到的任务级策略；
- a_t 是当前时刻要执行的动作；
- s_t 是当前状态；
- $\mathcal{D}_{\text{tr}}$ 是当前任务已收集到的少量交互经验，通常包含状态、动作、奖励及转移。

这一定义已经暗示了几个关键点。第一，任务描述不是显式给定的，而是隐含在数据集 $\mathcal{D}_{\text{tr}}$ 中。第二，隐藏状态或上下文表示必须能跨 episode 保持，否则代理无法在“探索 episode”里学到的信息带到“执行 episode”里。第三，元测试能否成功，依赖于测试任务是否来自与元训练相近的任务分布。

课程强调的泛化边界

Meta-RL 并不承诺“见到任何新任务都能适应”。它的承诺更接近于：如果新任务属于训练任务分布的同一家族，那么少量试错后可以显著加快学习。

讲者还回到 few-shot 图像分类与 LLM in-context learning，说明 Meta-RL 可以被看成“序列决策版的上下文学习”：监督学习里是“给几张样本图再分类”，RL 里则是“给几段交互轨迹再决定下一步动作”。

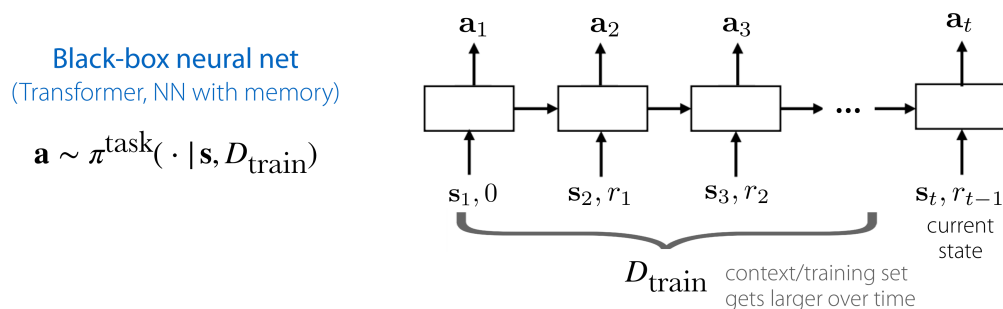
2.1 本章小结

Meta-RL 的形式化核心是任务分布、少量上下文经验，以及依赖上下文进行决策的策略。它讨论的不是如何在单一 MDP 里学到最优策略，而是如何在同分布新任务上更快适应。

3 Black-box Meta-RL 的做法

讲义后半段把重点放在所谓 black-box Meta-RL 上。所谓 black-box，意思是我们不手工设计显式的贝叶斯更新或解析任务推断器，而是直接用带记忆的神经网络，把历史交互序列作为输入，让网络自己学会从经验中归纳任务。

Black-Box Meta-RL: Overview



Question: How is this different from simply doing RL with a recurrent policy?

Reward is passed as input
(& trained across multiple MDPs)

Hidden state maintained
across episodes within a task!

16

图 2: Black-box Meta-RL 把历史 experience 当作上下文输入，通过带记忆的网络隐式推断任务。

从这个视角看，black-box Meta-RL 像是“把适应过程内化进网络前向传播里”。输入不再只有当前状态，还包括过去的状态、动作和奖励。于是，RNN、self-attention、1D convolution，甚至 deep set 这类可并行聚合上下文的结构，都可以承担“从上下文中提取任务信息”的职责。

课程总结了几类典型结构：

- RNN 类方法：通过隐藏状态跨时间累积任务信息，最接近“策略带记忆”。
- attention 或 transformer 风格方法：显式处理上下文序列，对较长上下文更灵活。
- deep set 聚合：把上下文嵌入后再池化，计算快且易并行。

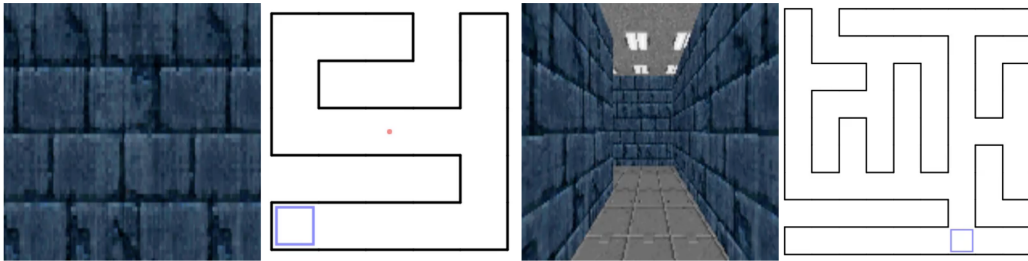
元训练过程则通常是：先采样一个任务，再在该任务上滚动若干 episode，把序列存进缓冲区，让优化器根据“后续 episode 回报是否提高”来更新整个系统。课程特别强调，奖励会作为输入喂给网络，同时网络会在同一任务的多个 episode 间保留隐藏状态，这正是它区别于普通 recurrent RL 的地方。

Meta-RL Example #1

From: Mishra, Rohaninejad, Chen, Abbeel. *A Simple Neural Attentive Meta-Learner*. ICLR 2018

Experiment: Learning to visually navigate a maze

- train on 1000 small mazes
- test on held-out small mazes and large mazes



20

图 3: 视觉迷宫导航实验展示了 black-box Meta-RL 的工作方式: 先探索, 再利用记忆快速到达目标。

在例子层面, 本讲给了三类代表性应用:

- 视觉迷宫导航: 在训练时看过大量小迷宫, 测试时在新迷宫里先探索、再直奔目标。
- 测试时计算 (test-time compute): 把数学题求解看作一类任务, 模型通过尝试不同推理策略来适应具体问题。
- 连续控制中的上下文变量方法: 如 PEARL, 用潜变量 z 表示任务, 再由上下文来估计后验。

讲者还插入了一个很重要的观察: 如果把经验集 $\mathcal{D}_{tr}$ 视作任务描述, 那么 Meta-RL 就能被理解成“多任务策略的一个推广版”。多任务策略接收显式任务 ID, 而 Meta-RL 则接收隐式的、由经验构成的任务描述。

3.1 本章小结

Black-box Meta-RL 的实质, 是让神经网络自己从过去的轨迹和奖励中学习任务推断。它的优势是通用、端到端; 代价是可解释性较弱, 而且当探索很难时, 优化会变得不稳定。

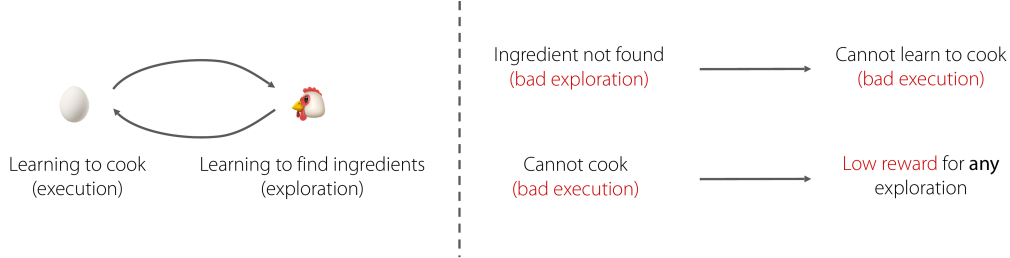
4 Meta-RL 中的探索难题

讲者随后把焦点拉回一个真正困难的问题: **如果测试时既需要探索, 又需要利用探索结果求解任务, 那么这种能力该怎么学?** 在迷宫、厨房、长走廊等问题中, 代理常常必须先做信息收集, 再根据收集到的信息去完成目标。

课程先介绍了端到端方案: 直接用任务最终回报来联合优化探索 episode 与执行 episode 的行为。它的优点很纯粹, 因为理论上它可以学到最优的探索-利用折中; 但缺点也很明显, 探索和执行强烈耦合, 容易出现训练信号极弱的局面。

Why is End-to-End Training Hard in This Example?

End-to-end approach: optimize exploration and execution episode behaviors end-to-end to maximize reward of execution



Coupling problem: learning exploration and execution depend on each other

—> can lead to poor local optima, poor sample efficiency

Liu, Ragunathan, Liang, Finn. *Decoupling Exploration and Exploitation for Meta-Reinforcement Learning without Sacrifices*. ICML 2021

图 4: 探索与执行的耦合问题: 如果既不会找食材也不会做饭, 那么两边都拿不到足够学习信号。

这一点在“新厨房做饭”的例子中尤其清楚。要学会做饭, 先得找到食材; 但若不会做饭, 即使找到了食材也拿不到高回报。于是探索和执行像“鸡生蛋、蛋生鸡”一样互相依赖, 端到端优化虽原则上正确, 却可能样本效率极差。

本讲给出的第二条路线, 是借助更结构化的探索策略。代表做法是 posterior sampling, 也就是 Thompson sampling 风格的方法。它先根据已有上下文估计任务后验, 再从后验中采样一个潜变量, 按“当前猜测的任务”去行动。

$$z \sim q(z \mid \mathcal{D}_{\text{tr}}), \quad a_t \sim \pi(a_t \mid s_t, z).$$

这里:

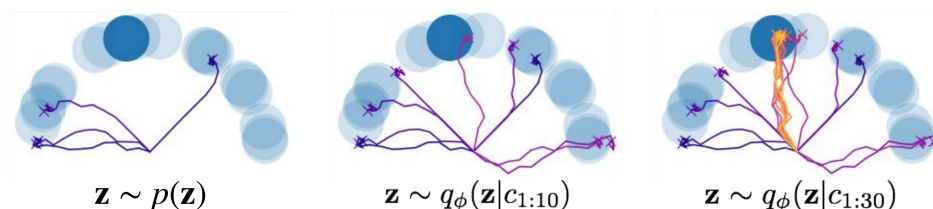
- z 是潜在任务变量;
- $q(z \mid \mathcal{D}_{\text{tr}})$ 是基于上下文经验得到的任务后验;
- a_t 是动作;
- $\pi(a_t \mid s_t, z)$ 是条件在状态与潜变量上的策略。

Solution #2: Leverage Alternative Exploration Strategies

2a. Use posterior sampling
(also called Thompson sampling)

PEARL (Rakelly, Zhou, Quillen, Finn, Levine. ICML '19)

- Learn distribution over latent task variable $p(\mathbf{z}), q(\mathbf{z} | \mathcal{D}_{tr})$ and corresponding task policies $\pi(\mathbf{a} | \mathbf{s}, \mathbf{z})$
- Sample $\mathbf{z}$ from current *posterior* and sample from policy $\pi(\mathbf{a} | \mathbf{s}, \mathbf{z})$



When might posterior sampling be bad? Eg. Goals far away & sign on wall that tells you the correct goal.

图 5: PEARL 用潜变量后验来驱动探索：先推断可能的任务，再按该任务对应策略行动。

这种思路以 PEARL 为代表。它不把探索完全交给一个隐式 recurrent policy，而是显式学习“上下文到潜变量后验”的映射，再让策略条件在该潜变量上。相较于完全端到端的方法，这更容易把“任务推断”和“执行策略”部分解耦。

本讲的边界提醒

Posterior sampling 并不自动解决一切问题。它缓解的是任务不确定性下的探索设计，但前提仍然是：训练阶段看过足够丰富的任务分布，并且上下文里确实包含区分任务所需的信息。

4.1 本章小结

探索是 Meta-RL 最困难的部分。端到端优化理论上最完整，却容易陷入耦合困境；后验采样等结构化方法牺牲了一些纯粹性，但往往能换来更好的可训练性与样本效率。

5 总结与延伸

这讲把 Meta-RL 的主题完整搭了起来。前半部分回答“它是什么”：Meta-RL 研究的是如何在任务分布上学会快速适应；中间部分回答“常见方法是什么”：black-box Meta-RL 用带记忆的神经网络把少量经验变成隐式任务描述；后半部分回答“真正难点是什么”：当测试时需要探索时，训练会面临严重的耦合问题。

如果把这讲与前一讲的多任务 RL 对照起来，可以得到一个很清楚的层级关系：

- 多任务 RL 假设任务描述符是已知的；
- 目标条件 RL 是其中一种特殊情形；
- Meta-RL 假设任务描述符未知，只能通过少量经验在线推断。

因此，Meta-RL 既和 few-shot learning 相通，也和 LLM 的 in-context learning 有共鸣。真正重要的不是术语，而是一个统一视角：模型是否能利用上下文快速重构“当前到底是什么任务”。

从研究角度看, 这讲留下了两条后续主线。第一条是 exploration, 本课程下一讲会专门展开, 讨论从 bandit 到大规模 MDP 的探索问题; 第二条是如何用更显式的结构, 如 posterior sampling、上下文变量、解耦探索与执行, 来提升 Meta-RL 的可训练性与稳定性。对读者而言, 理解这两条线, 基本就抓住了 modern Meta-RL 的核心张力。